

Xortools Software Guide

Stephen P. Jordan^{*1}, Noah Shutty^{†1}, and Tanuj Khattar¹

¹*Google Quantum AI*

Contents

1	Overview	2
2	Installation	2
3	Reproducing the Figures	3
3.1	Table of Approximations	3
3.2	Semicircle Plot	6
3.3	Simulated Annealing and Greedy Algorithm Plots	8
3.4	Convergence Plots	9
3.5	Approximation Comparison Plot	11
3.6	Bleichenbacher-Nguyen Plots	12
3.7	Degree Distribution Histogram	13
3.8	Information-Theoretic Limitations Plot	14
3.9	Plot of Simulated Annealing Performance on OPI	14
3.10	Resource Table	16
4	File Formats	16
4.1	Weighted Max-XORSAT	16
4.2	Max-LINSAT	17
4.3	PSOL	18
4.4	SOL	18
4.5	DEG	18
5	MacOS Compatibility	19
6	Code Catalog	19
7	Data Catalog	32
A	Binomial Moment Theorem	33
B	Decoding Failure	34
C	Timing Data Regarding Software Performance	36

^{*}stephenjordan@google.com

[†]shutty@google.com

1 Overview

The xortools software is a collection of tools used to carry out the research reported in [1]. We provide it here for others to build upon and to ensure the reproducibility of our results. The license is as reported in LICENSE.txt. This software is made available with no promises of support but if you have a question you should feel free to email us and we will do our best to answer it if possible.

2 Installation

The xortools software is written in C++20 and Python3. The C++ programs are comprised of two collections. The first collection can be compiled using the Makefile in the `xortools` directory. The second collection can be compiled using a tool called Bazel. Many of the C++ programs can be compiled with no dependencies, but some require the Eigen package, and some require the NTL number theory library and the GNU Multiprecision Library (GMP). A few require the Boost library. We used Eigen version 3.4.0, NTL version 11.5.1, GMP version 6.3.0, and Boost version 1.86.0. We compiled xortools on MacOS using clang and on linux using gcc. If you use Windows the easiest way to compile this code is probably through Windows Subsystem for Linux (WSL). The requirements for the Python scripts are listed in requirements.txt. In variants of Linux that use the apt package manager (such as Ubuntu), all dependencies other than NTL can be installed as follows.

```
sudo apt-get install -y software-properties-common
sudo add-apt-repository ppa:ubuntu-toolchain-r/test
sudo apt-get update
sudo apt-get install -y libeigen3-dev libgmp3-dev libboost-all-dev g++-13 gcc-13
```

On Ubuntu Linux, the Eigen headers will be, by default, placed in a different location than on Mac. If you don't want to modify the xortools Makefile, you can simply move the Eigen headers to the location expected by xortools, as follows.

```
cd /usr/include/eigen3
for eigenthing in *
do
    sudo mv $eigenthing ..
done
```

Next, Bazel can be installed as follows.

```
cd ~
wget https://github.com/bazelbuild/bazelisk/releases/download/v1.25.0/bazelisk-linux-amd64
chmod +x bazelisk-linux-amd64
sudo mv bazelisk-linux-amd64 /usr/local/bin/bazel
```

For the collection of C++ programs associated with a Makefile, the following should work:

```
cd ~/xortools
make check
make
```

If `make` doesn't work then you may still be able to compile specific tools from `xortools`. For example, if you only need to make the belief propagation decoder then you can just use the command `make load_decode`. For the collection of C++ programs associated with Bazel, the following should work.

```
bazel build src:all
```

To compile only one specific C++ program among those associated with Bazel, you can use commands like the following example.

```
bazel build src:anneal
```

For convenience, you may wish to carry out the following command and also add it to the end of `~/.profile`.

```
export PATH="$PATH:~/xortools/bin:~/xortools/bazel-bin/src"
```

3 Reproducing the Figures

In the following subsections we describe how to perform the computations that generate the data to be plotted for the figures and tables in the manuscript. The actual data and the Python scripts for plotting it are in `xortools/plots`.

3.1 Table of Approximations

In this subsection, we describe how to reproduce the following table.

Random Sparse max-XORSAT	
Algorithm	SAT Fraction
Tailored Heuristic (7 min $\times$ 1 core)	0.880
Long Anneal (73 hrs $\times$ 5 cores)	0.832
DQI + Belief Propagation	≥ 0.831
Prange's algorithm	0.812
Short Anneal (8 sec $\times$ 1 core)	0.764
Greedy Algorithm	0.666
AdvRand	0.554

For Random Sparse max-XORSAT, the performance numbers are obtained by computer experiment using a concrete instance. This instance is contained in the file:

```
xortools/20240812_instance/m50000n31216.tsv.
```

The format of this file is explained in §4.1. Each of the performance results listed in the right panel of table 1 can be obtained as follows.

DQI + Belief Propagation: The performance of DQI+BP is dictated by the performance of the Belief Propagation decoder, via the semicircle law in [1]. The xortools package contains multiple interfaces to Belief Propagation. One is built using `make`. The resulting executable is `xortools/bin/load_decode`. The other is built using Bazel. The resulting executable is `xortools/bazel-bin/src/load_decode`. The latter is slightly more full featured. We therefore recommend using the following command.

```
/xortools/bazel-bin/src/load_decode m50000n31216.tsv --min-ell 6350 --max-ell 6350
--shots 10000 --maxiter 10000
```

This will extract the matrix B from the max-XORSAT instance specified in `m50000n31216.tsv`, while ignoring the weights. (Since the weights are all $\pm\frac{1}{2}$ these can be captured by a corresponding vector $\mathbf{v} \in \mathbb{F}_2^m$ such that the max-XORSAT instance is $B\mathbf{x} \stackrel{\text{max}}{=} \mathbf{v}$.) The `load_decode` program will then generate 10,000 random errors $\mathbf{e} \in \mathbb{F}_2^m$, each of Hamming weight 6,350 and test for each one whether Belief Propagation correctly recovers it from the syndrome $B^T\mathbf{e}$. The fraction of the 10,000 errors that were incorrectly decoded by Belief Propagation will be written to the terminal as `error_rate`. These computations may take several hours to complete. Due to the random sampling, results vary slightly from trial to trial, in our sample we found that Belief Propagation succeeded on all but 9 of the 10,000 trials at $\ell = 6,350$, thus corresponding to error rate 0.0009.

According to the semicircle law from [1], the asymptotic value of $\langle s \rangle / m$ achieved by DQI when $\ell = 6,350$, $m = 50,000$, and $\epsilon = 0.0009$ is at least

$$\frac{\langle s \rangle}{m} \rightarrow \frac{1}{2} + \sqrt{\frac{\ell}{m} \left(1 - \frac{\ell}{m}\right)} - \epsilon = 0.832. \quad (1)$$

However, this neglects to take into account finite-size effects. To incorporate these, we should use the following equation from [1]

$$\mathbb{E}_{\mathbf{v}} \langle f \rangle \geq \left(\frac{\mathbf{w}^\dagger A^{(m,\ell,0)} \mathbf{w}}{\mathbf{w}^\dagger \mathbf{w}} - 2\epsilon(m+1) \right), \quad (2)$$

which provides a lower bound on expected value of the objective $\langle f \rangle$ in terms of our choice of the vector $\mathbf{w} = (w_0, \dots, w_\ell)$. Choosing $\mathbf{w}$ optimally yields

$$\frac{\mathbf{w}^\dagger A^{(m,\ell,0)} \mathbf{w}}{\mathbf{w}^\dagger \mathbf{w}} \rightarrow \lambda_{\max}^{(m,\ell)},$$

where $\lambda_{\max}^{(m,\ell)}$ is the principal eigenvalue of the $(\ell+1) \times (\ell+1)$ matrix called $A^{(m,\ell,0)}$ whose matrix elements are specified in [1]. Since f is the number of satisfied constraints s minus the number of unsatisfied constraint, and the total number of constraints is m we have $s/m = 1/2 + f/2$. Thus, the above equation yields

$$\frac{\langle s \rangle}{m} \geq \frac{1}{2} + \frac{\lambda_{\max}^{(m,\ell,0)}}{2m} - \epsilon. \quad (3)$$

For given m and ℓ , the quantity $\lambda_{\max}^{(m,\ell)}$ can be computed using `xortools/bin/tridiagonal.py`. Executing the command:

```
tridiagonal.py 50000 6350
```

yields the result 33199. Substituting $m = 50,000$, $\lambda_{\max}^{(50000,6437)} = 33199$, and $\epsilon = 0.0009$ into (3) yields $\langle s \rangle / m \geq 0.831$.

Simulated Annealing: The `xortools` package has several implementations of simulated annealing, but the implementation used to generate table 1 is the `anneal` executable, which is built using Bazel. When in the `xortools/20240812_instance/` directory, one can test simulated annealing on the instance using the command `anneal m50000n31216.tsv`. Or, for more informative output, run:

```
anneal --verbose m50000n31216.tsv
```

Here we have not added any additional command line arguments to override the default parameters, so simulated annealing will linearly increase inverse temperature β from 0 to 5 over the course of 10^6 sweeps. This instance has 5×10^4 variables and therefore 10^6 sweeps implies 5×10^{10} Metropolis updates. This may take a few hours to compute. Because simulated annealing is a randomized algorithm, the optima it finds will vary slightly from run to run.

Truncation Heuristic: Our implementation of the truncation heuristic (also known as Prange’s algorithm) is called `gaussheur`, which is compiled using `make`. It can be run using the command:

```
gaussheur m50000n31216.tsv.
```

Greedy Algorithm: Our implementation of greedy hillclimbing is called `greedy` and is compiled using `make`. It can be run using the command:

```
greedy m50000n31216.tsv.
```

Our greedy algorithm starts from a random assignment to the variables. So, one should expect the results to agree approximately with the result listed in the table in our manuscript, though not exactly.

AdvRand: Our implementation of the AdvRand algorithm from [2] is called `advrand`. It is compiled using `make`. It can be run using the command:

```
advrand m50000n31216.tsv
```

Tailored Heuristic: Our implementation of our tailored heuristic is implemented in the files `src/optimize/irreg_anneal_main.cpp` and `src/optimize/anneal.cpp`. It can be compiled using the command.

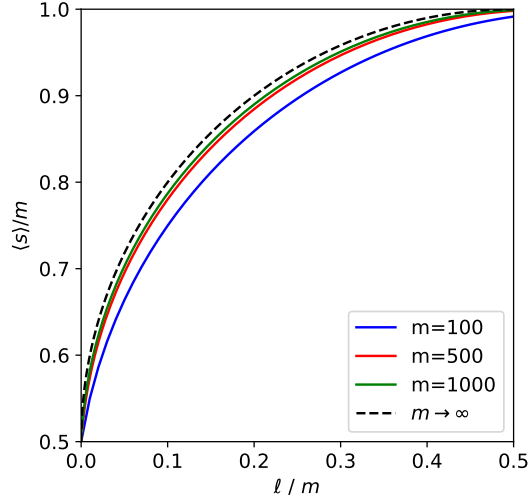
```
bazel build src:irreg_anneal
```

and run using the command

```
irreg_anneal m50000n31216.tsv --iterations 1000000
```

3.2 Semicircle Plot

In this subsection we describe how to reproduce the following plot.



Simply plotting the semicircle formula

$$\frac{\langle s \rangle}{m} = \frac{1}{2} + \sqrt{\frac{\ell}{m} \left(1 - \frac{\ell}{m}\right)} \quad (4)$$

doesn't require any C++ programs. However, this formula only captures the asymptotic behavior of DQI in the limit of large m with ℓ/m fixed. Here we also plot the exact behavior of $\frac{\langle s \rangle}{m}$ as a function of m for various moderately sized values of m . One way to compute the exact finite-size value of $\frac{\langle s \rangle}{m}$ is by finding the principal eigenvalue of the $(\ell+1) \times (\ell+1)$ matrix $A_{(m,\ell,0)}$ defined in our manuscript. However, as a sanity check against this method, we can use other, more direct methods to calculate $\frac{\langle s \rangle}{m}$ numerically. Two such more-direct methods are implemented in `polytools.hpp`. One can run these methods by first compiling via `make polyscratch` and then running the resulting executable `polyscratch`.

The executable `polyscratch` must be called with four command line arguments, as follows:

```
polyscratch m l d delta
```

Here, `m` is the number of constraints, `l` is the degree of the polynomial P , `d` is the distance of the code $C^\perp$, and `delta` is the failure rate of the decoder. For the purpose of producing figure 5, we can take `delta`=0.0 and `d`=21+1. Upon running `polyscratch`, the first line of output will contain three numbers: `m`, `l`, and $\langle s \rangle$. These are calculated using the method of Young polynomials, described below. After this line there will be many additional lines of output, which pertain to other ways of calculating $\langle s \rangle$ that are simpler but less numerically stable. These lines can be

ignored, as they merely serve as a sanity check by which the correctness of the Young polynomial method can be verified at small m .

The above is all that one needs to know to use `polyscratch`. The above plot can be reproduced by running `polyscratch` at various m and l and plotting the resulting $\langle s \rangle$. The remainder of this section describes the numerical methods by which `polyscratch` operates, for those who may wish to modify and extend the software. For this discussion, it is simpler to work in terms of s , the number of constraints satisfied, rather than t , the objective value. These two are related by $t = 2s - m$. Furthermore, whereas $f(\mathbf{x})$ is the objective function that returns the number of constraints satisfied minus the number unsatisfied, let

$$h(\mathbf{x}) = (1/2)(f(\mathbf{x}) + m) \quad (5)$$

be the function that returns the number of constraints satisfied by $\mathbf{x} \in \mathbb{F}_p^n$. Given a degree- ℓ polynomial $P_1(f)$, we can write down another degree- ℓ polynomial $P_2(h)$ such that $P_1(f(\mathbf{x})) = P_2(h(\mathbf{x}))$ for all $\mathbf{x}$. This is achieved simply by interpreting (5) as a linear change of variables. Since these polynomials have the same degree, we can find the optimal value achievable by DQI as a function of ℓ by finding an expression for the expected number of constraints satisfied by state $|P(h(x))\rangle = \mathcal{N} \sum_{\mathbf{x} \in \mathbb{F}_p^n} P(h(\mathbf{x})) |\mathbf{x}\rangle$ with general P , and then optimizing over choice of P .

By the Born rule, the expression for the expected number of constraints satisfied by the bit string obtained by measuring $|P(h)\rangle$ is

$$\langle s \rangle_P = \frac{\sum_{s=0}^m s W(s) |P(s)|^2}{\sum_{s=0}^m W(s) |P(s)|^2}, \quad (6)$$

where $W(s) = |\{\mathbf{x} \in \mathbb{F}_p^n : h(\mathbf{x}) = s\}|$. A general degree- ℓ polynomial P can be described by its coefficients $P(s) = \sum_{k=0}^{\ell} u_k s^k$. In terms of $u_0, \dots, u_{\ell}$ we can rewrite (6) as

$$\langle s \rangle_P = \frac{\sum_{i=0}^{\ell} \sum_{j=0}^{\ell} u_i u_j \sum_{s=0}^m W(s) s^{i+j+1}}{\sum_{i=0}^{\ell} \sum_{j=0}^{\ell} u_i u_j \sum_{s=0}^m W(s) s^{i+j}}, \quad (7)$$

By theorem 1,

$$\sum_{s=0}^m W(s) s^k = \sum_{s=0}^m \binom{m}{s} s^k \quad \forall k \in \{0, 1, \dots, d_{\min} - 1\}, \quad (8)$$

where $d_{\min}$ is the distance of $C^{\perp}$. Therefore, as long as $2\ell + 1 \leq d_{\min} - 1$ we can substitute (8) into (7) and obtain the following as an exact equality.

$$\langle s \rangle_P = \frac{\sum_{i=0}^{\ell} \sum_{j=0}^{\ell} u_i u_j \sum_{s=0}^m \binom{m}{s} s^{i+j+1}}{\sum_{i=0}^{\ell} \sum_{j=0}^{\ell} u_i u_j \sum_{s=0}^m \binom{m}{s} s^{i+j}}. \quad (9)$$

Equation (9) gives $\langle s \rangle_P$ as a ratio of quadratic forms in the vector $\mathbf{u} = (u_0, \dots, u_{\ell})$. We can rewrite these quadratic forms in matrix notation as follows. Let $A \in \mathbb{R}^{\ell+1 \times m+1}$ be the following matrix

$$A_{ij} = j^i \sqrt{\binom{m}{j}} \quad i \in \{0, 1, \dots, \ell\} \quad j \in \{0, 1, \dots, m\}. \quad (10)$$

Let D be the $(m+1) \times (m+1)$ diagonal matrix defined by $D = \text{diag}(0, 1, \dots, m)$. We can then express (9) as

$$\langle s \rangle_P = \frac{\mathbf{u}^\dagger [ADA^\dagger] \mathbf{u}}{\mathbf{u}^\dagger [AA^\dagger] \mathbf{u}}. \quad (11)$$

The optimal polynomial is then given by the vector $\mathbf{u} \in \mathbb{C}^{\ell+1}$ that maximizes this ratio of quadratic forms. As discussed in [3], the solution to this optimization problem is the largest solution λ of the following generalized eigenvalue problem:

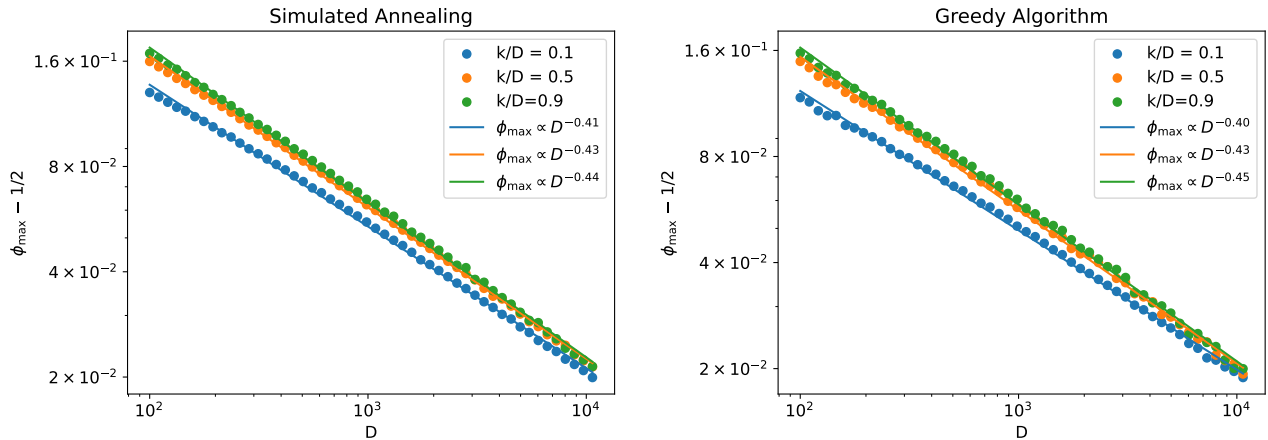
$$[ADA^\dagger] \mathbf{u} = \lambda [AA^\dagger] \mathbf{u}. \quad (12)$$

The generalized eigenvalue problem (12) can be solved numerically. This is one of the methods implemented in `polytools.hpp`. However, it is extremely ill-conditioned. We can avoid this problem by using a different basis for the space of degree- ℓ polynomials, instead of the basis of monomials that we implicitly used above. In [4], a simple recurrence relation is given to generate orthogonal polynomials with respect to any binomial measure. Using these polynomials as the basis vectors renders $AA^\dagger$ equal to the identity. With this choice of basis (12) becomes an ordinary eigenvalue problem, which turns out to be reasonably well-conditioned. This is the second method implemented in `polytools.hpp` which is called by the executable `polyscratch`.

We also note that, instead of the expected number of constraints satisfied we can set a target number of constraints T , and compute the probability of achieving target value T similarly to the above calculation. Instead of the diagonal matrix D in (12) we will have the diagonal projector $P_{ij} = \delta_{ij} \Theta(j \geq T)$. Using the Young basis, optimizing this probability then becomes an ordinary eigenproblem. Let $s_{\text{opt}}^{(1/2)}$ be the maximum value of T such that this probability is at least $1/2$. By numerically solving the eigenproblem one finds, unsurprisingly, that $s_{\text{opt}}^{(1/2)}/m$ converges in the limit of large m to the same semicircle law as $\langle s \rangle_{\text{opt}}/m$. However, unlike the above calculation of $\langle S \rangle_{\text{opt}}$, this calculation is not completely impervious to higher moments of $W(s)$, and in this context the replacement $W(s) \rightarrow \binom{m}{s}$ is only an approximation.

3.3 Simulated Annealing and Greedy Algorithm Plots

In this subsection we explain how to reproduce the following figure.



To produce this figure one must generate max-XORSAT instances from the Gallager ensemble and then find approximate optima for these instances using simulated annealing and the greedy algorithm. An instance of max-XORSAT from the Gallager ensemble can be produced using `random_regular`, which is built using `make`. An invocation takes the form

```
random_regular k D bsize.
```

This will produce an instance in which there are $D \times \text{bsize}$ constraints, each containing k variables, and $k \times \text{bsize}$ variables, each contained in D constraints. The vector $\mathbf{v}$, or equivalently the signs of the terms in the objective function, will be chosen uniformly at random. The output file will be in the format described in §4.1.

After producing an instance of max-XORSAT, one can run the greedy algorithm on it and simulated annealing algorithm on it. For example, one could run the following three commands.

```
random_regular 5 10 100
greedy instance_5_10_100.tsv
sawxor instance_5_10_100.tsv
```

The first command will write an instance into a file called `instance_5_10_100.tsv`. The next two commands run the greedy algorithm and simulated annealing on this instance, respectively. The number of iterations and annealing schedule used by `sawxor` are as described in [1] but can also be modified.

The full set of (k,D,bsize) triples in this plot are generated according to the following Python script.

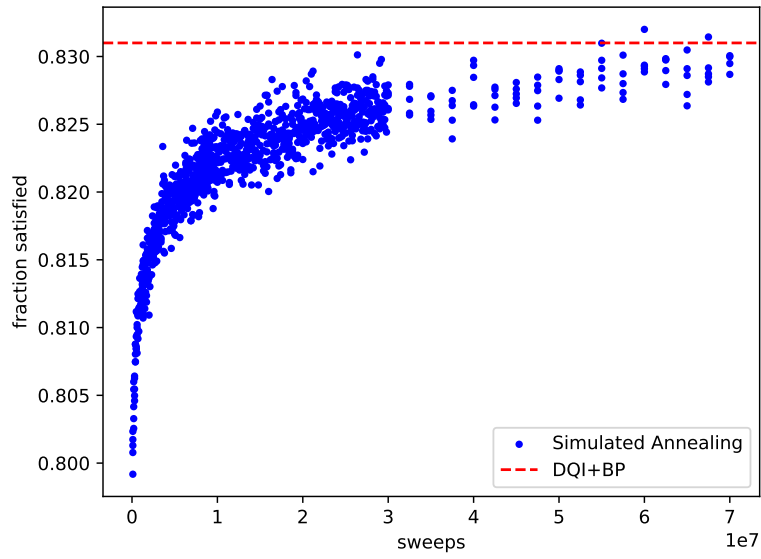
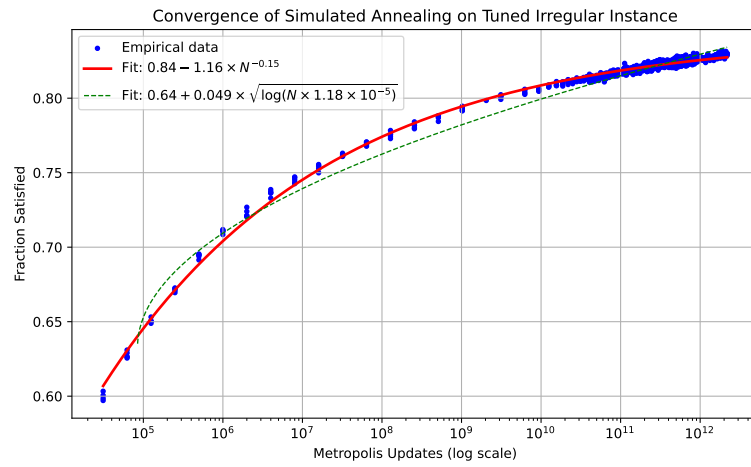
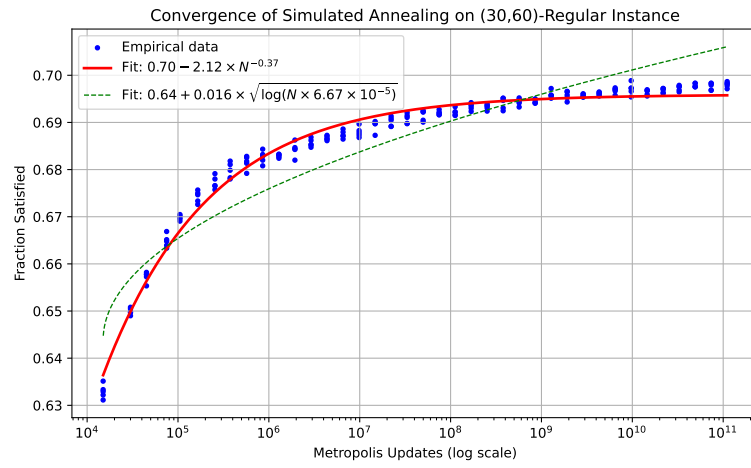
```
for KoverD in [0.1,0.5,0.9]:
    for s in range(0,50):
        D = round(100 * pow(1.1, s))
        k = round(KoverD*D)
        bsize = round(100000/k)
```

Running simulated annealing and the greedy algorithm on any individual resulting instance of max-XORSAT finishes quickly, but running them on all the instances takes a long time and is best done on a cluster computer.

The `xortools` package also contains another implementation of simulated annealing called `full_anneal`, which may run slightly faster than `sawxor` but that is not the implementation we used to generate this plot.

3.4 Convergence Plots

In this subsection we describe how to reproduce the following plots.



In these plots we show data about the convergence of simulated annealing as more sweeps are added. We apply this to two instances. The first is the tuned irregular instance:

`xortools/20240812_instance/m50000n31216.tsv`.

The second is a (60,30)-regular instance from the Gallager ensemble with $n = 15,000$. Such an instance can be generated by the command `random_regular 30 60 500`.

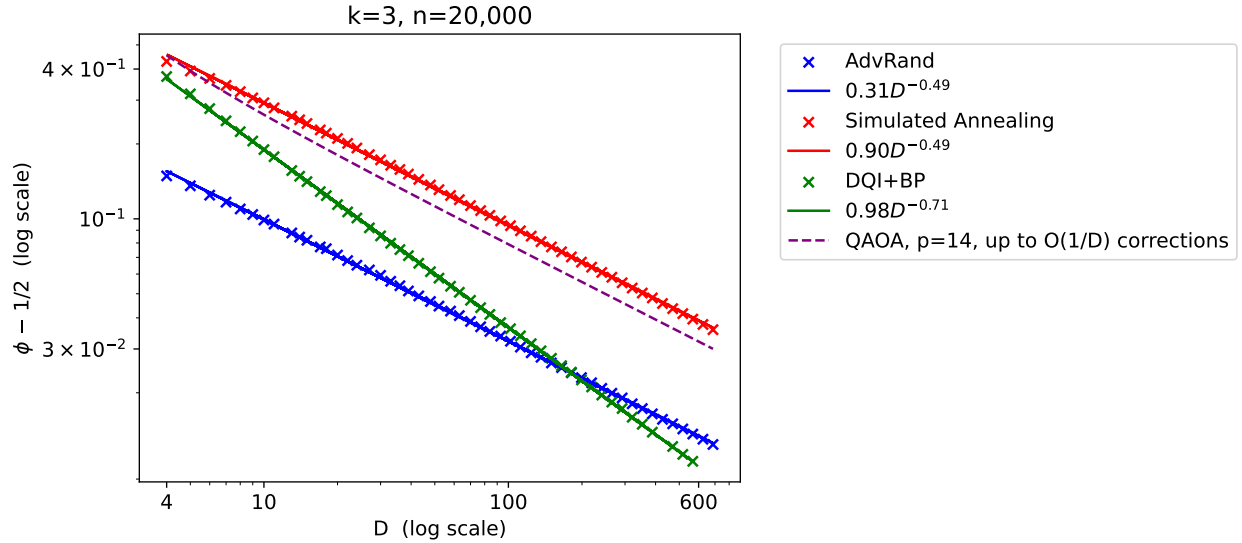
The first two plots show the convergence of simulated annealing versus sweeps on a linear scale and the third plot shows the convergence on a logarithmic scale. An individual data point would be produced using a command such as the following.

`bazel-bin/src/anneal --verbose --iterations 50000000 m50000n31216.tsv`

Collecting the data points required many such anneals with different numbers of sweeps, and was run on a cluster, using around 25 thousand CPU hours in total.

3.5 Approximation Comparison Plot

In this subsection we describe how to reproduce the following plot.



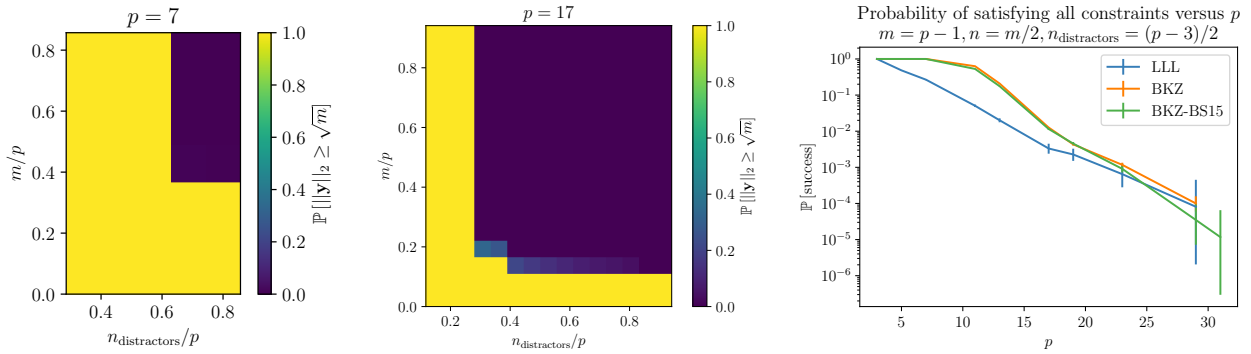
To generate this plot, we use `random_regular` to produce max-3-XORSAT instances from a constant- n family of instances that keeps the number of variables fixed as D varies. Then, we run simulated annealing via `sawxor`, the AdvRand algorithm via `advrand`, and Belief Propagation decoding via `load_decode`. We also compare against the approximation achieved by QAOA, which is obtained analytically in [5].

The performance of DQI+BP is then obtained by using the maximum number of decodable errors by BP, reported as `emax`, in the output of `load_decode` as ℓ in the asymptotic semicircle law (4). Since the instances are large, finite-size corrections should be negligible compared to the random fluctuations inherent in our experiments.

A subtlety worth mentioning is that `load_decode` defines `emax` as the maximum number of errors such that belief propagation succeeds in more than 60% of trials. Such high rates of decoding failure (*i.e.* up to 40%) are not compatible with the semicircle law from [1]; the resulting lower bound on $\langle s \rangle / m$ will be unusable. The reason we can tolerate such high rates of decoding failure in these instances is that, due to high values of D , Belief Propagation decoding fails already when the number of errors is smaller than half the distance of the code $C^\perp$. In this circumstance it is straightforward to prove that a high rate of decoding failure can at worst only moderately decrease the probability of obtaining a string of value $\langle s \rangle$. See theorem 4 in appendix B.

3.6 Bleichenbacher-Nguyen Plots

In this subsection we describe how to reproduce the following plots.



This figure studies the attack of Bleichenbacher and Nguyen [6] on the planted OPI problem. The planted OPI problem is the same as the OPI problem except that a planted solution $c \in C$ is first sampled. The sets $F_1, \dots, F_m$ are sampled such that that $c_i \in F_i$ for each $i \in [m]$.

Recall a bit of notation about OPI. The lists have size $|F_i| = r$ and we work over the field $\mathbb{F}_p$. The attack of [6] attempts to find a binary vector $\vec{\delta} \in \{0, 1\}^{m \cdot r}$ of length $\|\vec{\delta}\|_2 = \sqrt{m}$ and with one nonzero entry in each of the m groups of contiguous indices, as shown in Figure 1. The parity check matrix H is the parity check matrix of the Reed-Solomon code C , which is a Vandermonde matrix. The program `xortools/src/lattice/bleichenbacher_nguyen_main.cpp` provides a command-line interface to our C++ implementation of the attack. It can be built into an executable by running `bazel build src:bleichenbacher_nguyen`. It depends on the NTL library [7], which should be fetched, built, and included automatically by Bazel.

All of the command line interface arguments can be viewed by running

```
./bazel-bin/src/bleichenbacher_nguyen --help.
```

There are several parameters that must be specified for the attack. For example, either of `--lll` or `--bkz` must be supplied to specify the choice of basis reduction algorithm. It is possible to run the program with or without an instance file. Instances can be generated by the `plantrs` program: run `bazel build src:plantrs` then try running the program `./bazel-bin/src/plantrs --help` to see available options.

If the program is run without an instance file, it will collect data by sweeping p over prime numbers such that the program will have workable runtime. For LLL basis reduction, this is

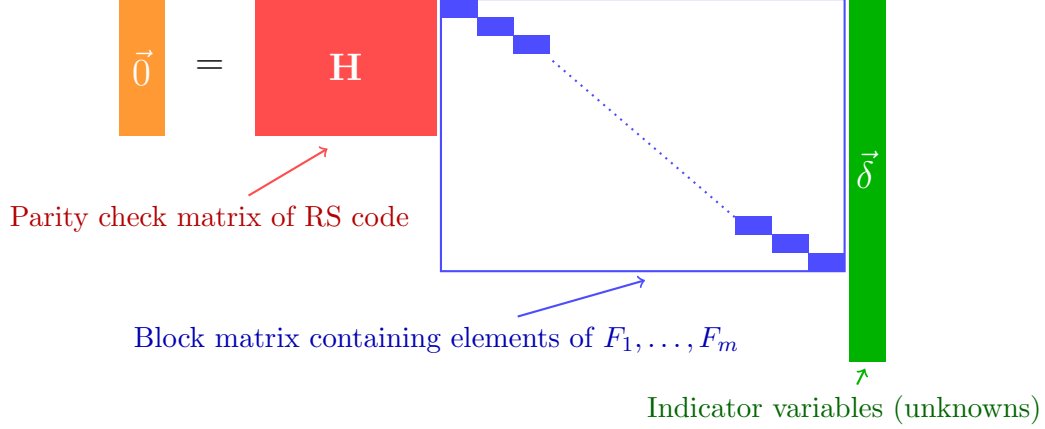


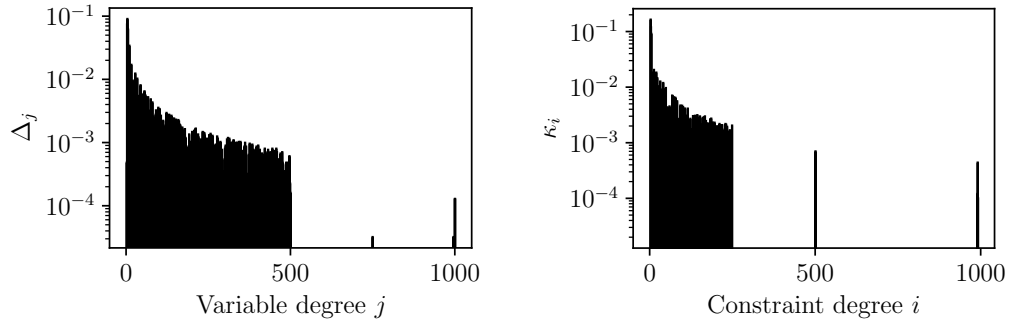
Figure 1: The attack of [6] computes the kernel over $\mathbb{F}_p$ of the linear system shown above. One of the kernel vectors is a binary solution $\vec{\delta}^*$ corresponding to the planted solution c where there is a single nonzero in each block at the position of the planted list element $c_i \in F_i$. This vector has length $\|\vec{\delta}^*\|_2 = \sqrt{m}$ which is sometimes short enough for lattice basis reduction techniques to find it.

between $p = 3$ and $p = 47$, whereas for BKZ it is between $p = 3$ and $p = 37$. But it will stop early depending on the parameters as various factors (e.g. the block size of BKZ) make the computation too slow for the larger primes.

The program writes a CSV file with statistics of the attack to standard output. After running this program on a cluster with different parameters, many csv files should be produced in the directory `bleichenbacher_nguyen_data/num_shots_data` with the extension `.lll.csv` or `.bkz.csv`, respectively. These can then be aggregated into one combined file by running `python src/py/aggregate_bleichenbacher_nguyen.py`. The data can then be plotted with the program `python src/py/plot_bleichenbacher_nguyen.py`.

3.7 Degree Distribution Histogram

In this subsection we describe how to reproduce the following plot as well as the associated table of degrees in the final appendix of [1].



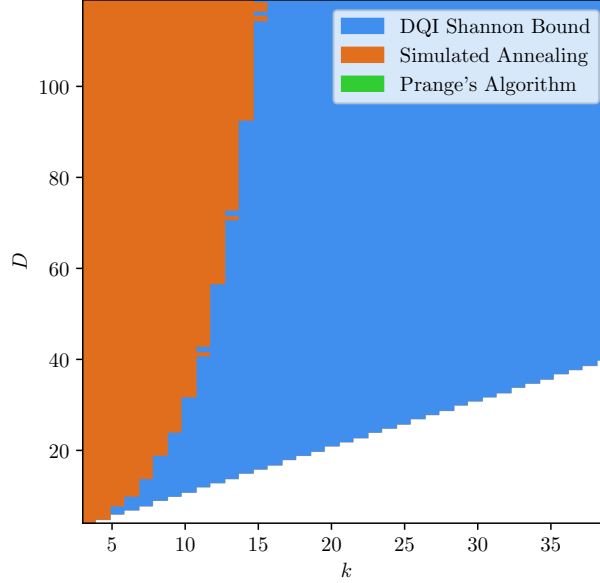
The degree distribution illustrated here is extracted from the instance file:

xortools/20240812_instance/m50000n31216.tsv.

The extraction of the data and formatting into latex tables is automated by the program `degrees`, which is built using `make`.

3.8 Information-Theoretic Limitations Plot

In this subsection we explain how to reproduce the following plot.



This plot considers degree- D max- k -XORSAT instances $B\mathbf{x} \stackrel{\max}{=} \mathbf{v}$ where $\mathbf{v}$ is uniformly random and B is drawn from the (k, D) -regular Gallager ensemble. The plot maps out regions in the (k, D) plane for which simulated annealing or Prange's algorithm exceed the upper bound that Shannon's limit implies for the value of $\langle s \rangle / m$ achievable by DQI with classical reversible decoding.

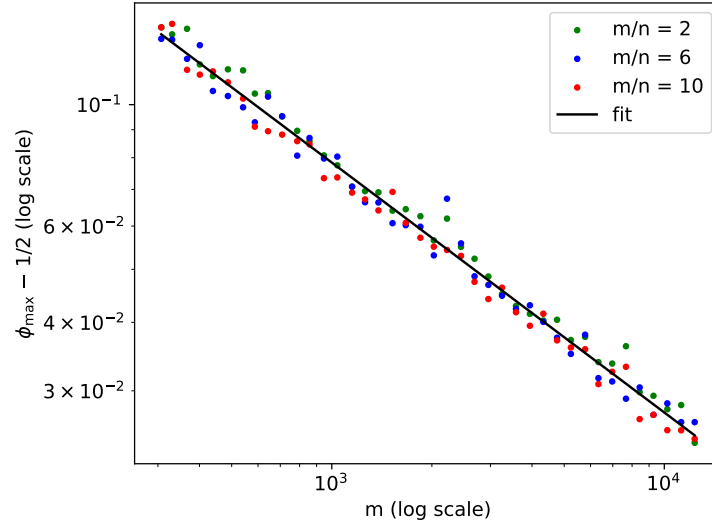
The simulated annealing data for this figure is generated using the executable `xortools/bazel-bin/src/anneal`, which is built using the command `bazel build src:anneal`. The instances are generated using the executable `xortools/bin/random_regular`, which is built using the command `make random_regular`. An example command to generate a single simulated data point used in the generation of this plot is:

```
anneal --verbose --iterations 500000 instance_3_4_666.tsv
```

The performance of Prange's algorithm and Shannon-limit DQI are predicted analytically, as described in [1].

3.9 Plot of Simulated Annealing Performance on OPI

In this subsection we explain how to reproduce the following plot.



This plot shows the performance of simulated annealing on the OPI problem. Each data point is obtained by first generating an OPI instance using `rsinstance`, and then running simulated annealing using `bigpanneal`. Each of these are C++ programs built by `make`.

The program `rsinstance` takes two command line arguments which indicate the desired number of variables and number of constraints. However, in OPI the number of constraints is equal to the modulus, which must be a prime. So `rsinstance` rounds up the given number of constraints m to the smallest prime $\geq m$. The number of variables is then scaled up by the same factor and rounded to the nearest integer, so as to match as closely as possible the ratio of constraints to variables specified by the input parameters. For example, running the command

```
rsinstance 10 20
```

will generate an OPI instance with $p = 23$ and $n = 12$ and output it to a file named `rs12_23.csv`, using the file format described in §4.2. One can then apply simulated annealing to this instance using the command

```
bigpanneal rs12_23.csv
```

The last line of the output to the terminal then specifies the number of constraints satisfied by the best solution found during the anneal. The list of values of m and n used to generate this plot is obtained by the following Python script.

```
for quotient in [2,6,10]:
    for power in range(0,40):
        m = round(300.0 * 1.1**power)
        n = round(m/quotient)
```

3.10 Resource Table

In this section we explain how to reproduce the following table.

$RS_p(N, K)$	BerlekampMasseyLFSR($n, \ell, \lceil \log_2 p \rceil$)	Toffoli	Clifford	Qubits
$RS_{67}(66, 34)$	(32, 16, 7)	999 850	5 153 696	907
$RS_{131}(130, 66)$	(64, 32, 8)	4 607 692	23 798 358	1912
$RS_{257}(256, 128)$	(128, 64, 9)	21 546 662	110 333 236	4101
$RS_{521}(520, 264)$	(256, 128, 10)	101 011 904	521 039 438	8850
$RS_{1031}(1030, 518)$	(512, 256, 11)	471 606 346	2 444 572 208	19 103
$RS_{2153}(2152, 1128)$	(1024, 512, 12)	2 186 280 548	11 380 033 666	41 132

For resource estimation of the Berlekamp-Massey algorithm, we use Qualtran [8] to implement and analyze quantum circuits for our reversible low qubit overhead implementation from Algorithm 2 of [1]. Specifically, we implement a “bloq” `BerlekampMasseyLFSR` which is present in the `xortools/resource_estimation/berlekamp_massey_lfsr_modp.py` file. To find the shortest linear feedback shift register (LFSR) of maximum length ℓ , for an input list of length n , with each element in $\mathbb{F}_p$, we can construct the bloq using `bloq = BerlekampMasseyLFSR.build(p, n,  $\ell$ )`. Once the bloq is constructed for a particular choice of parameters, we can use the analysis tools provided by Qualtran to verify correctness using classical and quantum simulations, get resource estimates in terms of gate counts and qubit counts and generate Cirq circuits containing only one and two qubit gates. For example, the following snippet shows how to generate the entries of the resource table.

```
from resource_estimation.berlekamp_massey_lfsr_modp import BerlekampMasseyLFSR
from qualtran.resource_counting import get_cost_value, QubitCount, QECGatesCost
params_small = [(32, 16, 67), (64, 32, 131), (128, 64, 257)]
params_large = [(256, 128, 521), (512, 256, 1031), (1024, 512, 2153)]
for (n, l, p) in [*params_small, *params_large]:
    bloq = BerlekampMasseyLFSR.build(p, n, l)
    gate_counts = get_cost_value(bloq, QECGatesCost())
    qubit_counts = get_cost_value(bloq, QubitCount())
    print(gate_counts.total_toffoli_only(), gate_counts.clifford, qubit_counts)
```

4 File Formats

4.1 Weighted Max-XORSAT

For instances of weighted max-XORSAT, we use files with the extension `.tsv` (Tab Separated Values). An important example is the instance `xortools/20240812_instance/m50000n31216.tsv`, which is described in [1]. On the first line of the instance file are two numbers separated by a tab. The first is the number of variables. The second is the number of constraints. Next there should be m additional lines in the instance file, where m is the number of constraints, as specified in the first line. Each of these lines defines a constraint. If we think of the variables as $x_0, \dots, x_{n-1} \in 0, 1$

the the objective function that we wish to maximize can be written as

$$f(x_0, \dots, x_{n-1}) = - \sum_{j=0}^{m-1} w_j \prod_{i \in C_j} (-1)^{x_j} \quad (13)$$

where w_j is the weight of the j^{th} constraint and C_j is set of indices of the variables contained in the j^{th} constraint. In the context of simulated annealing, it is more natural to think in terms of minimizing the “energy” function:

$$E(x_0, \dots, x_{n-1}) = \sum_{j=0}^{m-1} w_j \prod_{i \in C_j} (-1)^{x_j}, \quad (14)$$

which explains the presence of the minus sign in (13), which defines the meaning of the weights $w_0, \dots, w_{m-1}$. Thus, for example, second line of `xortools/20240812_instance/m50000n31216.tsv` which says:

0.5000 18867 24393 28093

indicates that the zeroth term in the energy function to be *minimized* is

$$\frac{1}{2} (-1)^{x_{18867}} (-1)^{x_{24393}} (-1)^{x_{28093}}. \quad (15)$$

In the language of max-XORSAT, this corresponds to the constraint

$$x_{18867} \oplus x_{24393} \oplus x_{28093} = 1. \quad (16)$$

If we run one of our annealers such as **sawxor** on such an instance, the solution $x_0, \dots, x_{m-1}$ will be output as a bit string, along with energy of the string, and the number of clauses satisfied by the string. The energy will equal to sums of the weights of the satisfied clauses minus the sums of the weights of the unsatisfied clauses.

4.2 Max-LINSAT

For instances of max-LINSAT, we use files with the extension `.csv` (Comma Separated Values). Other than any comment lines, in which the first character must be `#`, the first line of an instance file must consist of three numbers separated by commas. The first is the number of variables n , the second is the number of constraints m , and the third is the modulus p . (We will consider only prime p .) Each subsequent line specifies a constraint. The constraint is a concatenation of two lists. The first is a list of all values among $\{0, 1, \dots, p-1\}$ such that the function $f\{0, 1, \dots, p-1\} \rightarrow \{\text{false}, \text{true}\}$ evaluates to true. The second is a list of all nonzero coefficients in the argument to f . If the coefficient of x_i is $c_i \neq 0$ then the string $i : c_i$ will appear in the list. For example, suppose $p = 5$ and consider the line

1,3,0:3,2:1,4:4

This signifies that the constraint is

$$f(3x_0 + x_2 + 4x_4) \quad (17)$$

where

$$f(x) = \begin{cases} \text{true} & \text{if } x \in \{1, 3\} \\ \text{false} & \text{otherwise.} \end{cases} \quad (18)$$

4.3 PSOL

A file with the extension `psol` contains a solution to a max-LINSAT problem, *i.e.* an assignment $\mathbf{x} \in \mathbb{F}_p^n$ to the variables. Other than comments, which are lines whose first character is `#`, a `.psol` file consists of two lines. The first line is of the form `p,n`, where p is the modulus and n is the number of variables. The second line is a list of the values of the n variables, separated by commas. For example

```
7,15
2,6,6,5,1,6,0,3,6,0,0,6,6,6,0
```

is a valid `psol` file which specifies that the modulus is $p = 7$, the number of variables is $n = 15$, and the assignments to the variables are $x_0 = 2, x_1 = 6, \dots, x_{14} = 0$.

4.4 SOL

A file with the extension `sol` contains a solution to a max-XORSAT problem, *i.e.* an assignment $\mathbf{x} \in \mathbb{F}_2^n$ to the variables. A `.sol` file contains a single line, which is a bit string. For example if the content is `011` then $n = 3$ and $x_0 = 0, x_1 = 1, x_2 = 1$.

4.5 DEG

A file with the extension `deg` specifies the degree distribution of a Tanner graph defining a matrix B which then can be used to define a code $C^\perp = \{\mathbf{d} \in \mathbb{F}_2^m : B^T \mathbf{d} = \mathbf{0}\}$ or, together with a choice of $\mathbf{v} \in \mathbb{F}_2^m$ can be used to define an instance of max-XORSAT via $B\mathbf{x} \stackrel{\text{max}}{=} \mathbf{v}$. A `deg` file can contain comment lines, which must begin with `#`. Aside from comments, a `deg` file contains four lines, each of which consists of a list of numbers separated by whitespace. The first line lists a set of integers that represent degrees that a variable can have. The second line lists a set of floating point numbers that should add to one. These represent the fraction of variables that have each of the above-listed degrees. The third line lists a set of degrees that a constraint can have. The fourth and last line lists the fraction of constraints that have each degree. For example consider the file

```
# nodes
3 5 7 9 15 16
0.4 0.3 0.1 0.1 0.05 0.05
# checks
45 46
0.75 0.25
```

For the max-XORSAT instance, this specifies that 40% of the constraints have degree 3, *i.e.* contain 3 variables. Similarly, 30% of the constraints have degree 5, 10% of the constraints have degree 9, and so on. Next we see that 75% of the variables have degree 45, *i.e.* are contained in 45 constraints. Lastly, we see that 25% of the variables have degree 46. Since the average variable degree is 45.25 and the average constraint degree is 5.85, the ratio m/n of constraints to variables must be 7.73... For the code $C^\perp$ the top two lines refer to the degree distribution for bits in the codeword and the bottom two lines refer to the degree distribution of the parity checks.

5 MacOS Compatibility

There is a bug¹ in `rules_foreign_cc` which prevents us from using Bazel to build `gmp` on current Mac computers. Most of the tools can still be built as follows:

```
bazel build \
  //src:libsparse \
  //src:libutils \
  //src:libxorsat \
  //src:libldpc \
  //src:libbelief \
  //src:libirreg \
  //src:libdesign \
  //src:liboptimize \
  //src:load_decode \
  //src:density \
  //src:random_irregular \
  //src:random_kXOR \
  //src:design \
  //src:eval \
  //src:anneal \
  //src:chunk_anneal \
  //src:irreg_anneal \
  //src:strip_high_degree_clauses \
  //src:brute \
  //src:random_dense \
  //src:belief_test
```

6 Code Catalog

Not all of the programs contained in the Xortools Zenodo record are needed to reproduce the figures in [1]. Some were used for exploratory computations that ultimately did not contribute directly to the results reported in the paper. However, we describe these files exhaustively here, as they may prove useful for further research.

`adversarize.cpp`:

This was a program to generate the righthand side $\mathbf{v}$ of a max-XORSAT instance $B\mathbf{x} \stackrel{\text{max}}{=} \mathbf{v}$ in a way that is adversarial to the performance of simulated annealing rather than generating $\mathbf{v}$ uniformly at random, as is done by `random_regular`. To use `adversarize` one first generates a max-XORSAT instance in the format described in 4.1 and then provides it to `adversarize`. For example, one could run the commands

¹https://github.com/bazel-contrib/rules_foreign_cc/issues/1398

```
random_regular 10 20 30
adversarize instance_10_20_30.tsv
```

and this will produce a new file called `adv_instance_10_20_30.tsv` which is identical to `instance_10_20_30.tsv` except for the choice of $\mathbf{v}$ which is reflected in the weights of the constraints according to $\text{weight}_i = -\frac{1}{2}(-1)^{v_i}$.

`advrand_mp.cpp`:

This is the multithreaded implementation of the `advrand` algorithm for solving max-XORSAT, introduced in [5]. It takes as input an instance of max-XORSAT in the format described in §4.1.

`advrand.cpp`:

This is the single-threaded implementation of the `advrand` algorithm. The resulting executable has the same usage as `advrand_mp`.

`advrand.hpp`:

This header contains shared code used by both `advrand` and `advrand_mp`.

`baby_irreg.cpp`:

This is a program to generate random max-XORSAT instances where the constraints have a specified degree distribution. For example, we may wish to specify that among 200 constraints there are 50 that contain 5 variables and 50 that contain 20 variables. At present the degree distribution is hardcoded and the executable takes no command line arguments. To generate instances with different degree distributions one would edit the source code and recompile.

`bigp_anneal.cpp`:

This is an implementation of simulated annealing for max-LINSAT. It takes a single command line argument which is the input file specifying the max-LINSAT instance in the format described in §4.2. To alter the annealing schedule one would edit the source code and recompile.

`bitmatrix.cpp` and `bitmatrix.hpp`:

This is a high performance library that we wrote for performing linear algebra over the finite field $\mathbb{F}_2$, *i.e.* modulo 2. It packs the bits into 64-bit blocks and uses the intrinsic operations on the

CPU that in a single clock cycle will perform bitwise XOR, popcount (*i.e.* Hamming weight), etc. on blocks of 64 bits. The matrices are stored in row-major form so row operations are faster than column operations. There is no executable directly associated with these files. Rather, many of the other executables in the xortools package have these as dependencies. The C++ API for this library should be understandable by reading `bitmatrix.hpp`.

`brute.cpp`:

This is a program that, given an instance of max-XORSAT in the format specified in §4.1, will use brute force evaluation to produce a histogram showing how many among the 2^n assignments to the variables satisfy s of the m constraints for each value $s \in \{0, 1, \dots, m\}$. Of course, this can only be used for small n since the runtime scales exponentially in n .

`chebsilon.cpp`:

This program is similar in spirit to `polyscratch`, as described in §3.2. However, there are two major differences. First, instead of optimizing $\mathbf{w}$ to maximize the expected number of constraints satisfied, as is done by `polyscratch`, we here instead optimize $\mathbf{w}$ to maximize the probability that the number of constraints satisfied is greater than some specified threshold t . Secondly, rather than working the basis of Young polynomials, `chebsilon` works in the basis of Chebyshev polynomials.

`cperpscan.cpp`:

This program samples a sequence of parity check matrices from a given ensemble and for each one exhaustively computes the weight distribution of the resulting code. At the end, for each weight w the mean and standard deviation of the number of codewords with weight w is displayed. The exhaustive scanning is sped up through the use of Gray codes.

`degrees.cpp`:

This is used to automatically tabulate the degree distribution, *i.e.* the number of constraints containing a given number of variables, for a given max-XORSAT instance in the format described in §4.1.

`epsilon.cpp`:

This performs some estimates about the distribution of objective values achieved by DQI if one uses $P(f) = f^\ell$. It treats the general case of weighted max-XORSAT with arbitrary weights. Other than how many constraints of given weight magnitude the instance has, no other details of the instance are used. Nevertheless, one invokes its program by loading an instance file in the format specified in §4.1.

`erdos_renyi.cpp`:

This takes arguments m, n, p and then generates a max-XORSAT instance $B\mathbf{x} \stackrel{\max}{=} \mathbf{v}$ where B is an $m \times n$ matrix in which each entry independently chosen to be 1 with probability p and zero with probability $1 - p$. The vector $\mathbf{v} \in \mathbb{F}_2^m$ is chosen uniformly at random. The output is produced into a file which has the format specified in §4.1 and whose filename is constructed from n, m, p as follows: `ern_m_p.tsv`.

`evalist.cpp`:

The usage of `evalist` is as follows

```
evalist instance.csv solution.psol
```

Here, `instance.csv` is a file specifying an instance of max-LINSAT in the format described in §4.2 and `solution.psol` is a file containing a solution (*i.e.* an assignment to the variables) in the format described in §4.3. The output of `evalist` will be the number of constraints satisfied by the solution.

`evalxor.cpp`:

The usage of `evalxor` is as follows

```
evalxor instance.tsv solution.sol
```

Here, `instance.tsv` is a file specifying an instance of weighted max-XORSAT in the format described in §4.1 and `solution.sol` is a solution, *i.e.* assignment to the variables, in the format described in §4.4. The output will be $-f(\mathbf{x})$, where f is the weighted max-XORSAT cost function and $\mathbf{x}$ is the solution. For the instances of weighted max-XORSAT generated by `random_regular`, every weight is $\pm 1/2$. In this case the output will be $(1/2)(\#\text{UNSAT} - \#\text{SAT})$, where $\#\text{UNSAT}$ is the number of unsatisfied constraints and $\#\text{SAT}$ is the number of satisfied constraints.

`feldman_gurobi.cpp`:

The program `feldman_gurobi` requires two arguments. The first is the filename for an instance of weighted max-XORSAT in the format specified in §4.1. The second is a number of bit flip errors ℓ , which should be between 0 and m , the number of constraints in the weighted max-XORSAT instance. Given these inputs, `feldman_gurobi` will then generate a linear program representing a relaxation of the maximum likelihood decoding problem determined by the code $C^\perp$ dual to the max-XORSAT instance and a random error of Hamming weight ℓ . This is done using the method introduced in [9]. The program `feldman_gurobi` then uses Gurobi via its C++ API to solve the linear program. This will only work if Gurobi is installed on the machine on which `feldman_gurobi` is being run. Note that Gurobi is a commercial software package. To use free solvers for linear programs see `feldman_lp`.

`feldman_lp.cpp`:

The program `feldman_lp` requires two arguments. The first is the filename for an instance of weighted max-XORSAT in the format specified in §4.1. The second is a number of bit flip errors ℓ , which should be between 0 and m , the number of constraints in the weighted max-XORSAT instance. Given these inputs, `feldman_lp` will produce two output files. These files both contain a description of the same linear program, but use different formats, namely the LP format and the MPS format. The linear program encodes a relaxation of the maximum likelihood decoding problem determined by the code $C^\perp$ dual to the max-XORSAT instance and a random error of Hamming weight ℓ . The method for recasting decoding problems as linear programs is taken from [9]. The LP and MPS formats are standard and widely-used file formats for describing linear programs. The linear programs can then be loaded and solved using any standard solver, such as those available at neos-server.org.

`full_anneal.cpp`:

The program `full_anneal` finds approximate optima for instances of max-XORSAT using simulated annealing. Although it takes input in the `.tsv` format described in §4.1, which can store general weighted max-XORSAT instances, `full_anneal` only accommodates unweighted instances in which every constraint has weight exactly $\pm 1/2$. In this sense, `full_anneal` is slightly less general than `sawxor`. On the other hand, `full_anneal` may often run slightly faster and the code is perhaps slightly more readable. The program `full_anneal` requires only one input argument which is the filename of the instance. To modify the number of iterations or the temperature schedule one can edit the source code and recompile.

`gaussheur.cpp`:

The program `gaussheur` finds approximate optima of weighted max-XORSAT problems using Prange’s algorithm, as described in [1]. Only one command line argument is required, which is the input file specifying the weighted max-XORSAT instance in the format described in §4.1. In early drafts of our manuscript we referred to Prange’s algorithm as the Gaussian Heuristic, since it is based on Gaussian elimination.

`generate_inverses.cpp`:

The program `generate_inverses` is used to generate lookup tables for the multiplicative inverses in the finite fields $\mathbb{F}_p$ for all primes $p < 512$. This data is stored in `modpinverse.hpp`. So, there is no need to run this program again, unless `modpinverse.hpp` has become lost or corrupted.

`greedy.cpp`:

The program `greedy` finds an approximate solution to a weighed max-XORSAT problem using the greedy algorithm, as described in [1]. This program can be called with one argument or two. A command like the following

```
greedy instance_3_4_5.tsv
```

will load the weighted max-XORSAT instance specified in `instance_3_4_5.tsv` and apply greedy ascent from a uniformly random starting point. A command like the following

```
greedy instance_3_4_5.tsv solution.sol
```

will apply greedy ascent from the starting point specified in `solution.sol`. The file formats for the instance and the starting point are as specified in §4

`ldgmsat.cpp`:

The program `ldgmsat` tries to solve max-XORSAT problems using belief propagation. The belief propagation problem is formulated using the factor graph described in [10]. One invokes the program with one argument, which is the name of the file specifying the max-XORSAT instance in the format described in §4.1. In practice simulated annealing almost always works better.

`ldpc.cpp` and `ldpc.hpp`:

These files contain methods for loading and saving binary LPDC codes and decoding them using belief propagation. These files are not associated directly with any executable. Rather, they are dependencies of other executables such as `load.decode`.

`listsat.cpp` and `listsat.hpp`:

These files contain routines for loading, saving, and manipulating instances of max-LINSAT. These files are not directly associated with any executables. Rather they are used as dependencies by `evalist` and `pload.decode`. The file `maxlinsat.hpp` provides a different library of routines for loading, saving, and manipulating instances of max-LINSAT. The difference is that `listsat` uses the $\mathbb{F}_p$ inverse tables from `modpinverse.hpp` and `linsat` does not. Consequently, `listsat` is limited to prime p less than 512, whereas `linsat` does not have this limitation. On the other hand, because `linsat` does not implement inverses, it cannot compute a generator matrix for a code given its parity check matrix. Thus, `pload.decode` must be built upon `linsat` and therefore shares its limitation to prime $p < 512$.

`load_decode.cpp`:

This program loads an instance of max-XORSAT $B\mathbf{x} \stackrel{\max}{=} \mathbf{v}$ in the tsv file format defined in §4.1 and uses belief propagation to decode random bit flip errors applied to the code $C^\perp = \{\mathbf{d} \in \mathbb{F}_2^m : B^T\mathbf{x} = \mathbf{0}\}$. The program can be run as

`load_decode instance.tsv`

This will load `instance.tsv`, pick a random seed using the computer’s intrinsic source of randomness, and test the decoding ability of belief propagation with an increasing number of errors. First, it will try decoding with no errors, as a sanity check, then with one error, two errors, and so on until the success rate of the decoder drops below 1%. By default at error count h , the program will try 10 trials with 10 different error strings, which are uniformly chosen at random from all bit strings of length m and Hamming weight h . The program will report `emax` as the maximum number of errors for which the decoder achieved a success rate of at least 60%.

More detailed control over the operation of `load_decode` can be obtained by command line usage in one of the following three forms.

```
load_decode instance.tsv seed
load_decode instance.tsv seed errors
load_decode instance.tsv seed errors trials
```

If `seed` is specified this fixes the random seed instead of choosing it randomly using the computer’s internal source of entropy. If `errors` is specified, the tests are run only using random errors of this specified Hamming weight, instead of incrementing the Hamming weight. If `trials` is specified this overrides the default value of ten trials per Hamming weight. The optional arguments must be given in order. That is, there is no way to specify the number of errors without first specifying the seed, and there is no way to specify the number of trials without first specifying the seed and then specifying the number of errors.

`load_decode_ip.cpp`:

The program `load_decode_ip` decodes LDPC codes using an integer program via the method described in [11]. The integer programs are solved by calls to Gurobi via its C++ API. This will only work if one has Gurobi, which is a commercial software package.

`load_decode_lp.cpp`:

This is another LDPC decoder using linear programs. It uses a method from [9]. It depends on Gurobi for solving the linear programs.

`load_decode_R.cpp`:

This is another LDPC decoder using linear programs. This uses the polytope R defined in appendix II of [9]. It depends on Gurobi for solving the linear programs.

`lrandreg.cpp`:

The program `lrandreg` generates an instance of max-LINSAT, as described in [1], where the matrix B is taken from the mod- p analog of the Gallager ensemble and the lists $F_1, \dots, F_m$ are chosen uniformly and independently at random from all lists of size r . The usage is:

`lrandreg k D bsize p r`

where k is the number of variables contained in each constraint, and D is the number of constraints that each variables is contained in. The total number of variables is $k \times \text{bsize}$. The total number of constraints is $D \times \text{bsize}$. We work over the finite field $\mathbb{F}_p$, where p is a prime number given as a command line argument. Lastly, each constraint has a truth table where r of the p possible values evaluate to true. The resulting max-LINSAT instance is output to a file whose format is described in §4.2.

`maxlinsat.hpp`:

This file contains a number of routines for loading, saving, and manipulating instances of max-LINSAT. The programs `bigpanneal` and `rsinstance` depend on these routines.

`measure_bias.cpp`:

The program `measure_bias` runs simulated annealing many times and collects statistics about the approximate optima that are found. In particular, for each $i = 1, \dots, n$ it calculates the fraction of approximate optima in which the i^{th} bit is assigned to 1. Similarly, for each $j = 1, \dots, m$ it calculates the fraction of approximate optima in which the j^{th} constraint is satisfied. Also, it reports whether the same approximate optimum was found by more than one anneal. If so the program will output the message “Duplicate found!”.

`modpinverse.hpp`:

This header contains lookup tables regarding $\mathbb{F}_p$ arithmetic for small p . Specifically, it contains a lookup table of boolean values identifying, for each integer less than 512 whether it is prime. For each such prime $p < 512$ this file contains a lookup table for the multiplicative inverse map $x \rightarrow x^{-1}$ in $\mathbb{F}_p$.

`panneal.hpp`:

This header contains a templated implementation of simulated annealing that is used by `salist`.

`partial_anneal.cpp` and `partial_anneal_mp.cpp`:

The program `partial_anneal` optimizes an instance of max-XORSAT using a variant of simulated annealing that is adapted to instances with a highly irregular distribution over constraint degrees. Essentially, the idea is to first throw away the constraints with high degree and anneal the resulting instance. Then the high degree constraints are put back in and the solution is annealed some more. The program `partial_anneal_mp` is the multithreaded version of `partial_anneal`. From the command line, usage is the same as `sawxor`.

`pmatrix.cpp` and `pmatrix.hpp`:

These files contain a library for doing linear algebra over $\mathbb{F}_p$ where p is any prime smaller than 512. The C++ API is nearly identical to that used for the case $p = 2$ in `bitmatrix.cpp` and `bitmatrix.hpp`. These files are not directly associated with any executables but the programs `evalist`, `evalplin`, `listsat`, and `plinsat` all depend on this library.

`polyscratch.cpp`:

This program numerically computes the exact finite-size value of $\frac{\langle s \rangle}{m}$. The theory behind the numerical methods is described in §3.2. The executable `polyscratch` must be called with four command line arguments, as follows:

`polyscratch m l d delta`

Here, `m` is the number of constraints, `l` is the degree of the polynomial P , `d` is the distance of the code $C^\perp$, and `delta` is the failure rate of the decoder. One should take `delta=0.0` and `d=2l+1`, since the effect of limited code distance and nonzero decoding failure rate can now be handled by the theorems present in version 3 of [1].

`polytools.hpp`:

This header contains numerical routines used for computing the finite size effects in the semicircle law. These routines are used in `polyscratch`, `youngpoly`, `polyvec`, and `ansatz_test`.

`polytope.R.cpp`:

This program tests a subroutine needed by the linear-program decoder implemented in `load_decode.R`. Specifically it builds the polytope R defined in appendix II of [9].

`polyvec.cpp`:

This program outputs the vector of coefficients that define the optimal choice of polynomial P used by DQI to maximize $\langle s \rangle$. These are expressed not in the monomial basis but rather in the basis of Young polynomials defined in [4]. Usage is `polyvec m l`, where m is the number of constraints and l is the degree of P . Hence, the vector will have dimension $l+1$.

`randgauss.cpp`:

This program runs Prange's algorithm 100 times on a given instance of max-XORSAT. That is, a subset of the constraints are omitted such that the remaining system of $\mathbb{F}_2$ -linear equations is square instead of being overconstrained. Then the square system is solved using Gaussian elimination. The resulting bit string satisfies all of the retained constraints and usually about half of the omitted constraints. The subset of constraints that are omitted is a different random subset in each of the 100 trials. Then the best solution from the 100 trials is reported. The usage from the command line is the same as `sawxor`.

`random_regular.cpp`:

The program `random_regular` produces a max-XORSAT instance $B\mathbf{x} \stackrel{\max}{=} \mathbf{v}$ where $\mathbf{v} \in \mathbb{F}_2^m$ is chosen uniformly at random and B is drawn from Gallager's ensemble of $\mathbb{F}_2$ -matrices with k ones per row and D ones per column. Gallager's ensemble is described in appendix B of [1]. The basic usage is as follows.

`random_regular k D bsize`

This will produce a file containing the max-XORSAT instance whose name is constructed from the arguments according to the pattern `instance_k_D_bsize.tsv`. The file format is as described in §4.1. The program `random_regular` can also take additional arguments in order to generate constraints consistent with a planted solution rather than choosing $\mathbf{v}$ at random. See the code itself for documentation of this feature.

`random_tanner.cpp`:

The program `random_tanner` produces a max-XORSAT instance $B\mathbf{x} \stackrel{\max}{=} \mathbf{v}$ where $\mathbf{v} \in \mathbb{F}_2^m$ is chosen uniformly at random and B is the parity check matrix of a Tanner code. Basic usage is as follows.

`random_tanner k D n`

This will produce an instance with n variables where each constraint contains k variables and each variable is contained in D constraints.

`randreg.hpp`:

This header contains routines for generating max-LINSAT instances in which the matrix B has the same sparsity pattern as in the Gallager ensemble, but the nonzero entries are chosen uniformly from $\{1, \dots, p-1\}$, where p is the modulus of the max-LINSAT instance. The executables `lrandreg` and `prandreg` depend on this header.

`rsinstance.cpp`:

The program `rsinstance` generates an instance of max-LINSAT where the matrix B is the parity check matrix of a Reed-Solomon code. The lists $F_1, \dots, F_m$ are chosen to be random subsets of $\mathbb{F}_p$ of size $\lfloor p/2 \rfloor$. Usage of `rsinstance` is described in §3.9.

`salist.cpp`:

The program `salist` uses simulated annealing to solve instances of max-LINSAT for any prime modulus p below 512. The constraints can be arbitrary lists. The input file format is as described in §4.2.

`sawxor.cpp`:

The program `sawxor` solves instances of weighted max-XORSAT using simulated annealing. An example of usage is `sawxor instance_5_10_100.tsv`, where `instance_5_10_100.tsv` is a file specifying an instance of weighted max-XORSAT in the format described in §4.1. Such a file could be generated using the command `random.regular 5 10 100`. The number of iterations and annealing schedule used by `sawxor` are as described in [1] but can also be modified.

`tanatmis.cpp`:

The program `tanatmis` implements a linear-program-based decoder for LDPC codes defined in [11]. This program uses a commercial software package called Gurobi to solve the resulting linear programs, and will only work on a machine on which Gurobi is installed.

`test_xortools.cpp`:

The program `test_xortools` runs all the unit tests for the `xortools` software library. This program must be run from the `xortools` directory, because it will look for necessary data files in the `data/` subdirectory of the directory that it is run from. The program `test_xortools` gets run when you use the command `make check`. If all the unit tests pass then the final line of output will be `ALL TESTS PASSED`. Otherwise, you can examine the detailed output for specific failures. In addition to unit tests, `test_xortools` runs some performance-oriented regression tests. Some reference timing results are given in appendix §C.

`utils.cpp` and `utils.hpp`:

The files `utils.cpp` and `utils.hpp` provide a collection of useful basic primitives that most of the other programs in `xortools` use. The primitives can be understood by examining these two source files and reading the comments within.

`wscan.cpp`:

The program `wscan` exhaustively enumerates the weight distribution of certain codes, *i.e.* for each $h \in \{0, 1, \dots, m\}$, how many of the 2^n codewords have Hamming weight exactly h . The codes used are not from the Gallager ensemble. Rather, each row of the $n \times m$ generator matrix is chosen uniformly at random among bit strings of Hamming weight D , where D is given as a command line argument as follows: `wscan n m D`. If the argument D is given equal to m then the program will instead sample a generator matrix uniformly at random from all $2^{n \times m}$ matrices in $\mathbb{F}_2^{n \times m}$.

`xoropt.cpp` and `xoropt.hpp`:

The files `xoropt.cpp` and `xoropt.hpp` contain routines for loading and saving unweighted instances of max-XORSAT and finding approximate optima for these instances using simulated annealing and variants thereof. Although weights other than $\pm 1/2$ are not accepted, the file format used for input and output is our file format for weighted max-XORSAT, defined in §4.2. There are not any executables directly associated with these files, but they are used as dependencies by `adversarize`, `baby_irreg`, `full_anneal`, `measure_bias`, `partial_anneal_mp`, and `partial_anneal`.

`xorsat.cpp` and `xorsat.hpp`:

The files `xorsat.cpp` and `xorsat.hpp` contain routines for loading, saving, and manipulating weighted max-XORSAT instances in the format described in 4.1. Many programs in the `xortools` package depend on these routines.

`youngpoly.cpp`:

The program `youngpoly` uses the same mathematical machinery that is used by `polyscratch` and described in §3.2. However, it is put toward a slightly different purpose. Recall that `polyscratch` optimizes the degree- ℓ polynomial P to maximize the expected number of satisfied constraints $\langle s \rangle$. The program `youngpoly` considers a target number of satisfied constraints t , and then optimizes the degree- ℓ polynomial to maximize the probability that at least t constraints are satisfied by the bit string obtained from the final measurement of the DQI state in the computational basis. This will be done for each t between `tmin` and `tmax`, which are specified as command line arguments. The usage is:

`youngpoly m l tmin tmax listsize modulus`

where `m` is the number of constraints, `l` is the degree of P , and `tmin` and `tmax` specify the range of t to scan through. The argument `listsize` specifies the number of entries in the truth table for each clause that evaluate TRUE. We call this r in [1]. The argument `modulus` is what we call p in [1].

`belief/load_decode_main.cpp:`

This can be compiled by running `bazel build src:load_decode`. It is a wrapper around the same underlying implementation of belief propagation for binary LDPC codes as is used in `load_decode.cpp`. However, this wrapper has more command line interface features. To see a full list of options, run `./bazel-bin/src/load_decode --help`.

`belief/density_main.cpp:`

This provides an interface to our implementation of density evolution. Density evolution is a fast algorithm introduced in [12] for computing the error threshold of LDPC codes under belief propagation decoding.

`belief/random_dense.cpp:`

This provides a means of generating fully random (i.e., unstructured and non-sparse) instances of max-LINSAT optimization problems.

`design/design_main.cpp:`

This program optimizes degree distributions for irregular LDPC codes. Similar to [13], it uses a differential evolution genetic optimization algorithm. The codes can be optimized for error correction performance or for advantage over simulated annealing.

`irreg/random_irregular.cpp:`

This program generates irregular LDPC codes of a desired size using a specified degree distribution. Uses the sampling procedure of [13].

`lattice/bleichenbacher_nguyen_main.cpp:`

This provides an implementation of the attack in [6]. The use of this tool is described in more detail in Section 3.6.

`ldpc/macwilliams_transform_main.cpp`:

The weight distribution of a linear code and its dual are related by the MacWilliams identities [14]. This program provides a convenient tool to transform from the weight distribution of a code to that of its dual, using these identities along with the GNU multi-precision arithmetic library [15].

`optimize/anneal_main.cpp`:

This program provides an additional implementation of simulated annealing (similar to `sawxor.cpp`). There are more command-line options in this implementation and some slightly different optimizations.

`optimize/irreg_anneal_main.cpp`:

This program provides an interface to the modified annealing algorithm optimized for irregular max-XORSAT instances. As discussed in the main paper, irregular annealing downweights the signal from high-degree clauses early on in the annealing process (at high temperature). This is an example of a “fine-tuned algorithm” that can beat DQI on the ensemble of irregular instances.

`src/sparse/`:

This directory contains the implementation of an important `SparseMatrix` data structure used to represent the LDPC code parity check matrices in all the other software. This directory does not provide any user-facing tools, but this data structure is important for any follow-up work that interfaces the functionalities of the xortools repo at the code level.

7 Data Catalog

`20240812_instance/`:

This subdirectory contains two files. The file `m50000n31216.tsv` contains the tuned irregular max-XORSAT instance described in [1]. This is stored in the format described in §4.1. The file `m50000n31216.load_decode` contains an output log from `load_decode` applied to this instance, which shows the performance of belief propagation.

`data/`:

The `data/` subdirectory contains data used in the unit tests implemented by `test_xortools`, which are run by executing `make check` in the `xortools` directory.

`degree.distributions/`:

This subdirectory contains several files in the `.deg` format described in §4.5. These degree distributions were obtained using density evolution to obtain high performance by belief propagation while at the same time achieving high average degree for both variables and constraints.

`partial.win.instances/`:

This subdirectory contains some instances of max-XORSAT such that DQI+BP is competitive with simulated annealing. However, the strongest performance of DQI+BP relative to simulated annealing and the truncation heuristic is for the instance in the directory `20240812.instance/`.

A Binomial Moment Theorem

The following is a slight generalization of a fact that is well-known in the coding theory community, namely that the d^{th} moment of the weight distribution of a binary linear code is equal to the d^{th} moment of the binomial distribution, provided d is less than the distance of the code's dual. Here we consider the case of general p , although to understand the code in `xortools`, it is only necessary to use the case $p = 2$.

Theorem 1. Fix $B \in \mathbb{F}_p^{m \times n}$, whose rows are $\mathbf{b}_1, \dots, \mathbf{b}_m$ and fix $f_i : \mathbb{F}_p \rightarrow \{+1, -1\}$ for $i = 1, \dots, m$. Let $f : \mathbb{F}_p^n \rightarrow \mathbb{Z}$ be the max-LINSAT objective function given by $f(\mathbf{x}) = \sum_{i=1}^m f_i(\mathbf{b}_i \cdot \mathbf{x})$. Suppose that $\sum_{x \in \mathbb{F}_p} f_i(x) = 0$ for all $i = 1, \dots, m$, and that $\mathbf{b}_i$ is nonzero for all $i \in 1, \dots, m$. Define $h_i : \mathbb{F}_p \rightarrow \{0, 1\}$ by $h_i(x) = (f_i(x) + 1)/2$ and $h(\mathbf{x}) = \sum_{i=1}^m h_i(\mathbf{b}_i \cdot \mathbf{x})$, so that $h(\mathbf{x})$ is the number of satisfied constraints. Let $W(s) = |\{\mathbf{x} \in \mathbb{F}_p^n \mid h(\mathbf{x}) = s\}|$. Let $d_{\min}$ be the minimum distance of the code $C^\perp = \{\mathbf{y} \in \mathbb{F}_q^m \mid \mathbf{y}B = 0\}$. Then for all $k = 0, 1, \dots, d_{\min} - 1$,

$$\sum_{s=0}^m W(s) s^k = \frac{p^n}{2^m} \sum_{s=0}^m \binom{m}{s} s^k.$$

Proof. First, we observe that any $d_{\min} - 1$ rows of B are linearly independent. Indeed, if there were a linear dependency involving $0 < d < d_{\min} - 1$ rows, there would be some $\mathbf{y} \in \mathbb{F}_p^n$ that is d -sparse so that $\mathbf{y}B = 0$; but then $\mathbf{y} \in C^\perp$ and has weight $d < d_{\min}$, contradicting the minimum distance.

Now, suppose we choose $\mathbf{x} \in \mathbb{F}_p^n$ at random, and consider $\mathbf{c} = B\mathbf{x}$. Because any $d_{\min} - 1$ rows of B are linearly independent, this implies that the symbols of $\mathbf{c}$ are $(d_{\min} - 1)$ -wise independent,

over the randomness of the choice of $\mathbf{x}$.² This implies that the values $h_i(\mathbf{b}_i \cdot \mathbf{x}) = h_i(c_i)$ are also $(d_{\min} - 1)$ -wise independent. Moreover, for each i , we have that $h_i(c_i)$ is distributed as a Bernoulli-1/2 random variable. This follows from the fact that c_i is uniformly distributed in $\mathbb{F}_p$ (as $\mathbf{b}_i \neq \mathbf{0}$), and the assumption that $\sum_{x \in \mathbb{F}_p} f_i(x) = 0$. Thus, for any $0 \leq k \leq d_{\min} - 1$, we have

$$\begin{aligned} \sum_{s=0}^m W(s) s^k &= \sum_{\mathbf{x} \in \mathbb{F}_p^n} h(\mathbf{x})^k \\ &= p^n \mathbb{E}_{\mathbf{x} \in \mathbb{F}_p^n} \left(\sum_{i=1}^m h_i(c_i) \right)^k \\ &= p^n \mathbb{E}_{z_1, \dots, z_m \in \{0,1\}} \left(\sum_{i=1}^m z_i \right)^k. \end{aligned}$$

where the z_i are i.i.d Bernoulli-1/2 random variables. Thus, we have

$$\sum_{s=0}^m W(s) s^k = \frac{p^n}{2^m} \sum_{\mathbf{z} \in \{0,1\}^m} \left(\sum_{i=1}^m z_i \right)^k = \frac{p^n}{2^m} \sum_{s=0}^m \binom{m}{s} s^k,$$

as desired. $\square$

B Decoding Failure

We start by recalling the following fact from Chapter 9 of [16].

Fact 1. *Consider quantum states $|\psi_{\text{ideal}}\rangle$ and $|\psi_{\text{real}}\rangle$. For any quantum measurement (POVM) let p_{ideal} and p_{real} be the probability distribution over outcomes arising from the ideal and real states, respectively. Then, the fidelity between these probability distributions is lower bounded by the fidelity between the states. That is,*

$$\sum_i \sqrt{p_{\text{ideal}}(i) p_{\text{real}}(i)} \geq |\langle \psi_{\text{ideal}} | \psi_{\text{real}} \rangle|$$

With this in hand, we now prove the following theorem.

Theorem 2. *Let $|P(f)\rangle$ be the ideal DQI state produced using a decoder with zero failure rate on errors up to Hamming weight ℓ . Let $|P_\epsilon(f)\rangle$ be the state obtained by measuring the $|\mathbf{y}\rangle$ register after decoding, and postselecting on $\mathbf{0}$, i.e. decoding success. Suppose this outcome is reached with probability $1 - \epsilon$. Suppose that measuring $|P(f)\rangle$ yields a string $\mathbf{x}$ so that $f(\mathbf{x}) > t$ with probability $1 - \eta$. Then measuring $|P_\epsilon(f)\rangle$ in the standard basis yields a string $\mathbf{x}$ so that $f(\mathbf{x}) \geq t$ with probability at least $(4/5)(1 - \epsilon - \eta)^2$.*

Proof. As discussed in [1], having produced a state

$$\sum_{k=0}^{\ell} \frac{w_k}{\sqrt{\binom{m}{k}}} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ |\mathbf{y}|=k}} (-1)^{\mathbf{v} \cdot \mathbf{y}} |\mathbf{y}\rangle |B^T \mathbf{y}\rangle, \quad (19)$$

²Indeed, fix any set $S \subseteq \{1, \dots, m\}$ of $d \leq d_{\min} - 1$ positions. Then $\mathbf{c}|_S = B|_S \cdot \mathbf{x}$, and $B|_S$ has full row rank, which implies that $\mathbf{c}|_S$ is uniform in $\mathbb{F}_p^d$.

the next step is to apply a decoding algorithm in superposition. From the bit string $B^T \mathbf{y}$ in the second register the decoder produces $\mathbf{y}_{\text{dec}}$, its best guess at $\mathbf{y}$. It then bitwise XORs $\mathbf{y}_{\text{dec}}$ into the first register yielding $\mathbf{y} \oplus \mathbf{y}_{\text{dec}}$, which equals $\mathbf{0}$ if the guess is correct. We then measure the first register in the computational basis. If we obtain $\mathbf{0}$ then we conclude that decoding has succeeded and we proceed with the remaining steps of the DQI algorithm.

Let Π be the projector onto the set of bit strings $B^T \mathbf{y}$ such that the decoder succeeds in uncomputing $\mathbf{y}$. If we obtain $|\mathbf{0}\rangle$ in the first register and discard it then the (normalized) post-measurement state of the remaining register is

$$|\tilde{P}_\epsilon(f)\rangle = \frac{\Pi |\tilde{P}(f)\rangle}{\sqrt{\langle \tilde{P}(f) | \Pi | \tilde{P}(f) \rangle}} \quad (20)$$

where $|\tilde{P}(f)\rangle$ is

$$|\tilde{P}(f)\rangle = \sum_{k=0}^{\ell} \frac{w_k}{\sqrt{\binom{m}{k}}} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ |\mathbf{y}|=k}} (-1)^{\mathbf{v} \cdot \mathbf{y}} |B^T \mathbf{y}\rangle, \quad (21)$$

which is recognizable as the ideal state that we wish to produce. Conditioned on the measurement outcome $\mathbf{0}$, the fidelity between the state we actually produce and the ideal state we wish to produce is thus

$$\langle \tilde{P}(f) | \tilde{P}_\epsilon(f) \rangle = \frac{\langle \tilde{P}(f) | \Pi | \tilde{P}(f) \rangle}{\sqrt{\langle \tilde{P}(f) | \Pi | \tilde{P}(f) \rangle}} = \sqrt{\langle \tilde{P}(f) | \Pi | \tilde{P}(f) \rangle}. \quad (22)$$

By assumption, decoding succeeds with probability $1 - \epsilon$, which implies that $\langle \tilde{P}(f) | \Pi | \tilde{P}(f) \rangle = 1 - \epsilon$. Hence,

$$\langle \tilde{P}(f) | \tilde{P}_\epsilon(f) \rangle = \sqrt{1 - \epsilon} \quad (23)$$

By the unitarity of the quantum Fourier transform we see that the final state we produce also has fidelity $\sqrt{1 - \epsilon}$ with its corresponding ideal state $|P(f)\rangle$.

We next show that measuring such a state still yields a bit string of objective value at least t with $1/\text{poly}(n)$ probability provided $1 - \epsilon - \eta > 1/\text{poly}(n)$. Let σ be the success probability, *i.e.* the probability of obtaining a bit string with objective value at least t upon measuring the imperfect state. As long as σ is not exponentially small, a bit string of objective value at least t can be obtained using polynomially many trials of DQI. We wish to lower bound σ as a function of ϵ and η . For this purpose, we use fact 2.

Take the POVM in fact 2 to be the two-outcome measurement defined by P_t and $I - P_t$, where I is the identity and P_t is the projector onto the set of bit strings whose objective value is at least t . Then, theorem 1 yields

$$\sqrt{(1 - \eta)\sigma} + \sqrt{\eta(1 - \sigma)} \geq \sqrt{1 - \epsilon}. \quad (24)$$

The function x^2 is monotonically increasing on the domain $\mathbb{R}^+$. Therefore, we can square both sides of (24) to obtain the following equivalent inequality.

$$(1 - \eta)\sigma + \eta(1 - \sigma) + \sqrt{\eta(1 - \eta)\sigma(1 - \sigma)} \geq 1 - \epsilon \quad (25)$$

Since $\sigma, \eta \in [0, 1]$ we have $(1 - \eta)\sigma \leq \sigma$, $\eta(1 - \sigma) \leq \eta$, and $\sqrt{\eta(1 - \eta)} \leq \frac{1}{2}$. Substituting these into (25) yields the following looser but still valid inequality

$$\sigma + \eta + \frac{1}{2}\sqrt{\sigma(1 - \sigma)} \geq 1 - \epsilon, \quad (26)$$

which rearranges to

$$1 - \epsilon - \eta \leq \sigma + \frac{1}{2}\sqrt{\sigma(1-\sigma)}. \quad (27)$$

Under the condition $1 - \epsilon - \eta \geq 0$ we can square both sides to obtain an equivalent inequality:

$$(1 - \epsilon - \eta)^2 \leq \sigma^2 + \sigma\sqrt{\sigma(1-\sigma)} + \frac{1}{4}\sigma(1-\sigma). \quad (28)$$

We next re-express the righthand side as $\sigma(\sigma + \sqrt{\sigma(1-\sigma)} + (1/4)(1-\sigma))$ and observe that the factor $\sigma + \sqrt{\sigma(1-\sigma)} + (1/4)(1-\sigma)$ is maximized at $\sigma = 4/5$ with value $5/4$. Thus (28) yields $(1 - \epsilon - \eta)^2 \leq (5/4)\sigma$, or in other words

$$\sigma \geq \frac{4}{5}(1 - \epsilon - \eta)^2. \quad (29)$$

□

The preceding theorem is phrased in terms of the failure probability ϵ of the measurement in which we postselect onto decoding success. In the case that the decoder fails on a nonzero fraction of the error syndromes and $2\ell \geq d^\perp$ one cannot easily deduce ϵ from the empirical performance of classical decoders. However, when $2\ell < d^\perp$ one can upper bound ϵ by the failure rate of the classical decoder on uniformly random syndromes of weight ℓ as we now show.

Theorem 3. *Let ϵ_k be the failure rate of a classical decoder for $C^\perp$ on uniformly random syndromes of Hamming weight k . Let $d^\perp$ be the distance of $C^\perp$. If $2\ell < d^\perp$ then the probability that DQI achieves postselection onto measurement success is $\epsilon = \sum_{k=0}^{\ell} w_k^2 \epsilon_k$. In the (commonly occurring) case that ϵ_k increases monotonically with k then one has the upper bound $\epsilon < \epsilon_\ell$.*

Proof. The first claim, that $\epsilon = \sum_{k=0}^{\ell} w_k^2 \epsilon_k$ is obtained by examining equation 19 and noting that in the case $2\ell < d^\perp$, the bit strings $B^T \mathbf{y}$ and $B^T \mathbf{y}'$ for any $\mathbf{y}$ and $\mathbf{y}'$ with Hamming weight at most ℓ are always distinct. The second claim then follows from the first using the fact that $p_k = e_k^2$ is a probability distribution. □

Together, theorems 2 and 3 imply the following.

Theorem 4. *Let ϵ_k be the probability that a classical decoder Dec fails when given a uniformly random syndrome of Hamming weight k . Consider the output from DQI+Dec algorithm where $2\ell < d^\perp$. Suppose that measuring the ideal DQI state (i.e. that which would be obtained with zero decoding failures) yields a string $\mathbf{x}$ so that $f(\mathbf{x}) > t$ with probability $1 - \eta$. If ϵ_k increases monotonically with k for $k \leq \ell$ then, conditioned on successful postselection in DQI+Dec, the final string $\mathbf{x}$ sampled will satisfy $f(\mathbf{x}) \geq t$ with probability at least $(4/5)(1 - \epsilon_\ell - \eta)^2$. Furthermore, the postselection will succeed with probability at least $1 - \epsilon_\ell$.*

C Timing Data Regarding Software Performance

As of April 14, 2025, when I run `test_xortools` on my M2 MacBook Pro, I get the following timing results. If your timings are substantially worse you may wish to experiment with compiler flags, etc.

Gathering timing data. (All times in milliseconds.)
n = 1000
two random generations: 0
two determinants: 11
two matrix-matrix products: 9
n = 2000
two random generations: 0
two determinants: 68
two matrix-matrix products: 46
n = 3000
two random generations: 1
two determinants: 175
two matrix-matrix products: 139
n = 4000
two random generations: 3
two determinants: 375
two matrix-matrix products: 291
two evaluations and 6400 differential evaluations: 3384 microseconds

Gathering p_timing data. (All times in milliseconds.)
n = 100
two p_random generations: 0
two p_determinants: 2
two p_matrix-matrix products: 0
n = 200
two p_random generations: 0
two p_determinants: 18
two p_matrix-matrix products: 1
n = 300
two p_random generations: 0
two p_determinants: 62
two p_matrix-matrix products: 3
n = 400
two p_random generations: 0
two p_determinants: 146
two p_matrix-matrix products: 9

Gathering p_mixed timing data. (All times in microseconds.)
n = 1000
two random generations: 1752
pmatrix times pvector (dense): 862
pvector times pmatrix (dense): 853

```

dense pmatrix times sparse pvector: 4
sparse pvector times dense pmatrix: 3
n = 2000
two random generations: 7203
pmatrix times pvector (dense): 5412
pvector times pmatrix (dense): 3712
dense pmatrix times sparse pvector: 12
sparse pvector times dense pmatrix: 7
n = 3000
two random generations: 16884
pmatrix times pvector (dense): 17596
pvector times pmatrix (dense): 7934
dense pmatrix times sparse pvector: 25
sparse pvector times dense pmatrix: 10
n = 4000
two random generations: 28826
pmatrix times pvector (dense): 23960
pvector times pmatrix (dense): 14186
dense pmatrix times sparse pvector: 54
sparse pvector times dense pmatrix: 16
Two plinsat evaluations and 20 differential evaluations: 9 microseconds

```

References

- [1] Stephen P. Jordan, Noah Shutty, Mary Wootters, Adam Zalcman, Alexander Schmidhuber, Robbie King, Sergei V. Isakov, and Ryan Babbush. Optimization by decoded quantum interferometry. *arXiv:2408.08292v3*, 2024.
- [2] Boaz Barak, Ankur Moitra, Ryan O’Donnell, Prasad Raghavendra, Oded Regev, David Steurer, Luca Trevisan, Aravindan Vijayaraghavan, David Witmer, and John Wright. Beating the random assignment on constraint satisfaction problems of bounded degree. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2015)*, volume 40 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 110–123, 2015. arXiv:1505.03424.
- [3] Stephen P. Boyd and Lieven Vandenberghe. *Convex optimization*. Cambridge University Press, 2004.
- [4] Tzay Y. Young. Binomial-weighted orthogonal polynomials. *Journal of the ACM (JACM)*, 14(1):120–127, 1967.
- [5] Joao Basso, Edward Farhi, Kunal Marwaha, Benjamin Villalonga, and Leo Zhou. The quantum approximate optimization algorithm at high depth for MaxCut on large-girth regular graphs and the Sherrington-Kirkpatrick model. In François Le Gall and Tomoyuki Morimae, editors, *17th Conference on the Theory of Quantum Computation, Communication and Cryptography (TQC 2022)*, volume 232 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 7:1–7:21, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

- [6] Daniel Bleichenbacher and Phong Q. Nguyen. Noisy polynomial interpolation and noisy Chinese remaindering. In *International Conference on the Theory and Applications of Cryptographic Techniques*, pages 53–69. Springer, 2000.
- [7] Victor Shoup. NTL: A library for doing number theory. <https://libntl.org/>. Accessed April 9, 2025.
- [8] Matthew P. Harrigan, Tanuj Khattar, Charles Yuan, Anurudh Peduri, Nouredin Yosri, Fionn D. Malone, Ryan Babbush, and Nicholas C. Rubin. Expressing and analyzing quantum algorithms with qualtran. *arXiv:2409.04643*, 2024.
- [9] Jon Feldman, Martin J. Wainwright, and David R. Karger. Using linear programming to decode binary linear codes. *IEEE Transactions on Information Theory*, 51(3):954–972, 2005.
- [10] Alexandros G. Dimakis, Martin J. Wainwright, and Kannan Ramchandran. Lower bounds on the rate-distortion function of ldgm codes. In *2007 IEEE Information Theory Workshop*, pages 650–655. IEEE, 2007.
- [11] Akin Tanatmis, Stefan Ruzika, Horst W. Hamacher, Mayur Punekar, Frank Kienle, and Norbert Wehn. A separation algorithm for improved LP decoding of linear block codes. *IEEE Transactions on Information Theory*, 56(7):3277–3289, 2010.
- [12] Thomas J. Richardson and Rüdiger L. Urbanke. The capacity of low-density parity-check codes under message-passing decoding. *IEEE Transactions on information theory*, 47(2):599–618, 2002.
- [13] Thomas J. Richardson, Mohammad Amin Shokrollahi, and Rüdiger L. Urbanke. Design of capacity-approaching irregular low-density parity-check codes. *IEEE transactions on information theory*, 47(2):619–637, 2001.
- [14] Florence Jessie MacWilliams and Neil James Alexander Sloane. *The theory of error-correcting codes*, volume 16. Elsevier, 1977.
- [15] Torbjörn Granlund. *GNU MP 6.0 Multiple precision arithmetic library*. 2015.
- [16] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2010.